

PRISM

POLICE · REPORTING · INTELLIGENCE · SYSTEM · MANAGEMENT

Solution & Architecture Brief — prepared for infrastructure sizing, hosting and deployment planning.

Why this document exists. It gives the DevOps/infrastructure engineer everything needed to choose a **cost-efficient server configuration and a simple hosting setup** for PRISM. It describes the system **as it exists today** (a working pilot-grade application), with a short forward-looking section so the chosen setup keeps a little headroom for the next few months — without over-provisioning for a full-scale rollout that is not yet in play.

What PRISM is

PRISM is a **web-based crime-record and reporting platform for the police hierarchy**. Head Constables and Station House Officers digitise records (FIRs, arrests, PCR calls, missing persons, unidentified bodies) at the police-station level; those records then flow **up a review-and-approval chain** — Station → Sub-division → District → City HQ — with every step audited. On top of the record store sit compilation, analytics dashboards, saved filters, and PDF/Excel report generation.

Functionally it is a fairly standard three-tier web application: a single-page browser front-end, a JSON API server, and a relational database — plus a message broker for background work and a report worker. There is nothing exotic in the runtime; the goal for hosting is **reliable, low-cost, and easy to operate**.

Scale context — read this before sizing

Pilot

CURRENT DEPLOYMENT
STAGE

~15–20

POLICE STATIONS IN NEAR-
TERM ROLLOUT

Dozens

CONCURRENT USERS
EXPECTED (NOT
THOUSANDS)

225 / 23

PS / DISTRICTS AT
EVENTUAL FULL SCALE
(LATER PHASE)

The full Delhi Police org tree is already modelled in config (23 districts, 92 sub-divisions, 225 police stations, ~2,855 beats), but **the system today is a functional pilot**. Traffic is measured in **dozens of concurrent officers** doing form entry and report generation — not high-throughput public traffic. **Size the infrastructure for the pilot with modest headroom**, not for the 225-station end state (that scale belongs to a later hardening phase and would be re-architected with search/replication at that point).

Current maturity — what is and isn't there yet

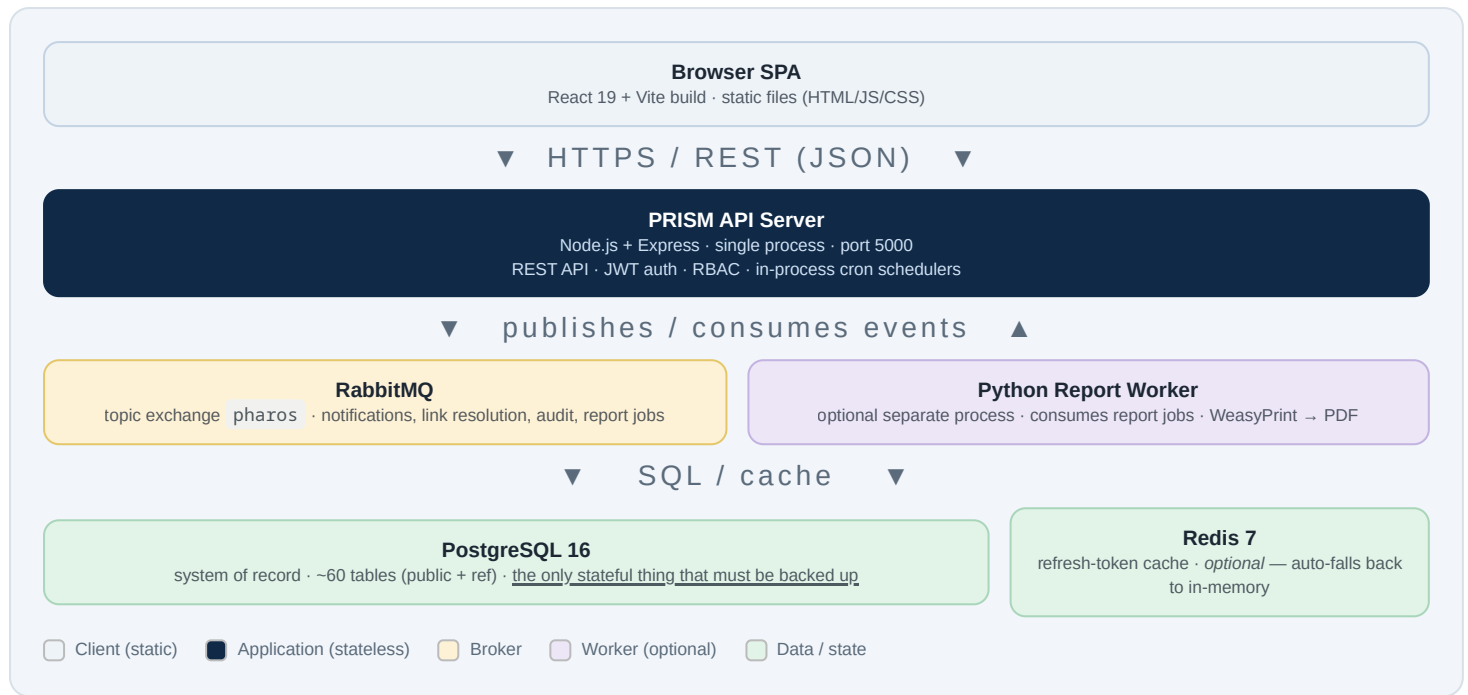
✓ **In place today**

- Full working backend API + React front-end
- PostgreSQL schema (migrations), seed & ref-data loaders
- RabbitMQ event bus + background handlers
- JWT auth & role-based access control
- PDF/Excel report generation
- `docker-compose` for local infra (DB / broker / cache)

◇ **Not built yet — DevOps to define**

- Production `Dockerfile` s / container images
- CI/CD pipeline (no `.github/workflows` yet)
- Reverse proxy / TLS termination (e.g. Nginx/Caddy)
- Process manager for the Node app (PM2/systemd/container)
- Backup schedule & off-host retention
- Log shipping / monitoring stack

The existing `docker-compose.yml` spins up Postgres, RabbitMQ and Redis for **local development only**. It is a good starting reference for what backing services are needed, but it is not a production topology.



How to read this for hosting

- **The front-end is just static files.** After `vite build` it is a folder of HTML/JS/CSS — serve it from any static host, the reverse proxy, or a CDN. It needs no server runtime of its own.
- **The API server is stateless.** All durable state lives in Postgres (and transient token state in Redis). The Node process can be restarted or replaced freely; it holds nothing that survives a restart.
- **PostgreSQL is the crown jewel.** It is the single component whose loss is unrecoverable. Everything in the backup/DR conversation is really about Postgres.
- **RabbitMQ** carries background work (notifications, cross-record link resolution, audit fan-out, report-job dispatch). If it's down, the API still serves requests; background side-effects just queue up / pause.
- **Redis is optional.** The code detects it and cleanly falls back to an in-memory token cache if it's absent or unreachable. Fine to skip for a single-instance pilot; add it when running more than one API instance.
- **The Python worker is optional and separate.** It exists for heavier report rendering. The Node app can also generate PDFs itself (via bundled Chromium), so the pilot can run without the Python worker at all.

Bottom line for topology: one application process (Node), one database (Postgres), one broker (RabbitMQ), an optional cache (Redis) and an optional report worker (Python). For a pilot this **comfortably fits on a single modest VM**, with a clean path to split the database out to a managed service later.

Runtime components

Component	Tech / version	Role	Class	State
Front-end SPA	React 19, Vite 8, Tailwind 4, Ant Design, Recharts	Browser UI for all roles. Built to static assets; served by proxy/CDN.	Core	Stateless
API server	Node.js (ES modules), Express 4	All business logic, REST API, auth, RBAC, workflow engine, in-process cron jobs. Single process, port 5000 .	Core	Stateless
PostgreSQL	16 (alpine)	System of record — ~39 public tables + 21 ref lookup tables. Knex-managed migrations.	Core	Durable ★
RabbitMQ	3.x (management)	Event bus (topic exchange pharos): notifications, link resolution, audit fan-out, report-job dispatch.	Core	Durable
Redis	7 (alpine)	Refresh-token store. Optional — code falls back to in-memory cache if not present.	Optional	Ephemeral
Python report worker	Python 3, pika, SQLAlchemy, pandas, WeasyPrint	Consumes report jobs from RabbitMQ, renders formatted PDFs. Optional / separate process.	Optional	Stateless
Chromium (bundled)	via Puppeteer 25	Headless browser the Node app launches on-demand to render PDF reports. Bundled with the npm package.	Core*	On-demand

★ Postgres is the only component whose data loss is unrecoverable. Core* = required only if PDF reports are used (they are a shipped feature, so treat Chromium's system libraries as required).

Explicitly NOT part of the runtime

Some libraries appear in the dependency manifest but are **not active** in the current build. Calling them out so you don't provision infrastructure that isn't needed:

Looks like it needs...	Reality	Notes
MongoDB (mongoose in deps)	Not used	Vestigial from an earlier design. The system is 100% PostgreSQL. Do not provision MongoDB.
Keycloak / SSO (keycloak-connect)	Off by default	Auth is JWT today. Keycloak only activates if KEYCLOAK_URL is set; otherwise it cleanly falls back to JWT. A near-future option, not a current dependency.
Object storage (S3 / MinIO)	Removed	File attachments were permanently removed from scope. No blob store needed. Generated report files land on local disk (REPORTS_DIR).
Elasticsearch	Not yet	Planned only for a much later high-scale phase. Not in the current system.

One packaging note for the DevOps engineer. The Node app renders PDFs with a bundled **Chromium**, and the optional Python worker uses **WeasyPrint**. Both pull in native system libraries (fonts, libnss3 , cairo/pango, etc.). If you containerise, the base image must include these or PDF generation fails at runtime — this is the single most common "works locally, breaks in prod" trap here.

Processes to run

Process	Command	What it does · notes
API server (required)	node index.js	The one long-running app process. On boot it connects to Postgres, runs a config auto-load, connects to RabbitMQ, starts event handlers, then listens on port 5000. Also runs three in-process cron schedulers (audit-chain verification, warehouse sync, scheduled reports) — so no separate cron service is needed.
Front-end (build step)	npm run build	Produces static assets once at deploy time. No runtime process — served as files.
Python report worker (optional)	python main.py	Long-running consumer for heavier report rendering. Skip for a minimal pilot; add when report volume grows.

Single-process simplicity. Background jobs (notifications, link resolution, audit, scheduled reports) run **inside the API process** via RabbitMQ handlers and node-cron. There is **no separate scheduler or queue-worker service to deploy** on the Node side. A process manager (PM2, systemd, or a container restart policy) to keep `node index.js` alive is all that's required.

Network ports

Service	Port	Exposure	Notes
API server	5000	Behind proxy	Put a reverse proxy (Nginx/Caddy) in front for TLS + to serve the SPA and route <code>/api</code> .
Front-end (dev only)	5173	Dev machine	Vite dev server. In production the SPA is static — this port is not used.
PostgreSQL	5432	Private only	Never expose publicly. App-to-DB on a private network/localhost.
RabbitMQ (AMQP)	5672	Private only	App-to-broker traffic.
RabbitMQ (management UI)	15672	Admin only	Handy dashboard — keep behind VPN/allowlist, not public.
Redis	6379	Private only	Optional; private network only.

Configuration (environment variables)

All config is via env vars (loaded from `backend/.env`). The full set is small:

NODE_ENV — <code>production</code>	PORT — API port (default <code>5000</code>)
DATABASE_URL — Postgres connection string	RABBITMQ_URL — AMQP connection string
REDIS_URL — optional; omit to use memory fallback	JWT_SECRET / JWT_REFRESH_SECRET — strong random
JWT_ACCESS_EXPIRES — e.g. <code>15m</code>	JWT_REFRESH_EXPIRES — e.g. <code>7d</code>
FRONTEND_URL — for CORS allowlist	REPORTS_DIR — disk path for generated files
STARTUP_AUTOLOAD — sync config on boot (<code>true</code>)	KEYCLOAK_URL — leave unset (JWT mode)

Security must-dos before go-live: replace the default `JWT_SECRET / JWT_REFRESH_SECRET` with strong random values; set strong Postgres and RabbitMQ credentials (the compose defaults `postgres/postgres` and `pharos/pharos123` are dev-only); keep all data-store ports private.

Where state lives

Store	Durability	Backup priority	What's in it
PostgreSQL	Must persist	Critical	All records, users, hierarchy, audit hash-chain, workflow ledger, config. The backup target that matters.
RabbitMQ	Should persist	Medium	Durable queues for in-flight background jobs. Losing it drops queued side-effects, not core data. Give it a persistent volume.
Report files on disk	Regenerable	Low	Generated PDF/CSV/XLSX under REPORTS_DIR . Reproducible from DB — a persistent volume is nice-to-have, not essential.
Redis	Ephemeral	None	Refresh tokens only. Safe to lose (users re-authenticate).

Backup strategy, in one line: a scheduled `pg_dump` (or managed-Postgres automated snapshots) shipped off-host is 95% of your DR story. Persistent volumes for Postgres and RabbitMQ; the rest is disposable.

Database footprint

The schema is compact and normalised: **~60 tables total** (39 application + 21 reference lookups). Row volume for a pilot is **small** — thousands to low tens-of-thousands of records, not millions. Reference data (offence codes, station codes, etc.) is loaded once from bundled config files. **Storage need is dominated by the OS, Node/Chromium install and logs, not by the database itself** at this stage.

First-time setup & deploy sequence

One-time database bring-up (from backend/)

```
npm run db:migrate → create schema (Knex migrations)
npm run sync-config → load form/workflow/report config from config/
npm run load-ref → load org hierarchy + reference lookup tables
npm run db:seed → seed baseline users (one per role)
```

Per-release deploy (typical flow)

- 1. Build front-end: `npm run build` → publish static assets to proxy/CDN
- 2. Install backend deps: `npm ci --prefix backend` (pulls Chromium)
- 3. Run any new migrations: `npm run db:migrate`
- 4. (Re)start the API process under the process manager
- 5. (Optional) start/restart the Python worker

On boot the API also auto-syncs config from the repo's `config/` folder (`STARTUP_AUTOLOAD=true`), so config changes ship with the code rather than through manual DB edits.

Observability & health (current state)

- **Logging:** structured logs via Winston to stdout — ready for a log collector; nothing proprietary.
- **Health:** the app fails fast and exits if Postgres or RabbitMQ can't be reached at startup, so a restart-on-failure policy (container/systemd/PM2) doubles as a crude health gate. A dedicated `/health` endpoint + metrics is a reasonable small add for DevOps.
- **No monitoring/APM wired yet** — greenfield for whatever lightweight stack you prefer.

These are starting-point recommendations to inform your sizing — you own the final cost/provider trade-off. The design goal is **cheap and simple for a pilot, with a clean upgrade path**.

★ Recommended for the pilot — single VM, all-in-one

Run everything on one Linux VM: the Node API, Postgres, RabbitMQ, (optional) Redis and the report worker, with Nginx/Caddy in front for TLS and to serve the SPA. Simplest to operate, lowest cost, and comfortably handles dozens of concurrent officers.

Resource	Suggested	Why
vCPU	4 vCPU	Node + Postgres share; Chromium PDF renders are CPU-bursty. 2 works, 4 gives headroom.
RAM	8 GB	Postgres cache (~2–3 GB) + Node (~0.5–1 GB) + RabbitMQ (~0.5–1 GB) + Chromium spikes (~0.5–1 GB/render) + OS.
Disk	80–100 GB SSD	OS + Node/Chromium install (heavy) + DB + logs + generated reports, with room to grow.
Bandwidth	Modest	Internal police traffic, not public web scale.

Alternative — split tier (when you want managed Postgres)

Slightly higher cost, less operational burden for the database, natural next step as usage grows:

Piece	Size	Notes
App VM (Node + worker + broker)	2 vCPU / 4 GB / 40 GB SSD	Stateless app tier; easy to resize or duplicate later.
Managed PostgreSQL 16	2 vCPU / 4 GB / 50 GB	Offloads backups, patching, point-in-time recovery to the provider.
RabbitMQ	Small (self-host or managed)	Can stay on the app VM for the pilot; externalise only if needed.
Redis (optional)	Smallest tier / skip	Only worthwhile once you run more than one API instance.
Static SPA	CDN / object host / proxy	Pennies — it's just files.

Cost-efficiency levers, ranked. **(1)** Start single-VM — one bill, one thing to operate. **(2)** Skip Redis and the Python worker initially; both are optional and the app degrades gracefully without them. **(3)** Serve the SPA as static files from the proxy — no separate front-end server or its cost. **(4)** Right-size at 4 vCPU / 8 GB; scale *up* the single VM before scaling *out* — the app is stateless, so a bigger box is the cheapest next step.

Scaling behaviour (so sizing choices are informed)

- **Scales up cleanly:** the API is stateless, so vertical scaling (more CPU/RAM) is the easy, cheap lever for the pilot.
- **Scales out later:** to run multiple API instances behind a load balancer, **add Redis** (so refresh tokens are shared, not per-instance memory) and point them at the same Postgres/RabbitMQ. The code already supports this.
- **The database is the eventual bottleneck**, not the app — which is why a managed Postgres with read replicas is the natural growth path well before the app tier needs much.

What's coming in the next few months (leave a little room)

Sizing for the pilot is right — but keep these in mind so the setup isn't a dead-end. None require provisioning now; they inform "don't pick the absolute smallest box / most rigid setup".

Near-term change	Infra impact	Headroom to keep
Keycloak / MFA auth	New service (or managed IdP)	Auth already has a Keycloak fallback path. If it lands, you'll add one more small service or a managed identity provider — leave a little RAM/CPU margin.
Extra review tiers (JCP / SCP chain)	More records & queries	Modest DB/query growth as more of the hierarchy comes online. The 4 vCPU / 8 GB recommendation absorbs this.
More report templates + scheduled reports	More PDF rendering load	This is where the optional Python worker earns its place — move report rendering off the API process so form entry stays snappy.
More police stations onboarded	More users & rows	Handled by scaling the single VM up; add Redis + a second API instance if concurrency climbs.

Explicitly out of near-term scope — do NOT provision for these now: Elasticsearch, the full 225-station load, high-availability multi-region, and object storage. These belong to a later hardening phase and would be planned separately. Provisioning them today is wasted spend.

Open questions for the DevOps engineer

Hosting

- Cloud provider / on-prem / government data-centre constraints?
- Data-residency or compliance rules (police data)?
- Preferred TLS / domain / reverse-proxy standard?

Operations

- Containers vs. plain VM + systemd/PM2 — your call.
- Backup cadence & off-host retention target?
- Monitoring/log stack you want us to emit toward?

DevOps go-live checklist

- ☐ Provision VM (≈4 vCPU / 8 GB / 80–100 GB SSD) or split tier per \$06
- ☐ Install: Node.js LTS, Postgres 16, RabbitMQ, (opt) Redis, (opt) Python 3 + WeasyPrint deps
- ☐ Ensure Chromium system libraries present (fonts, libnss3, cairo/pango) for PDF generation
- ☐ Reverse proxy (Nginx/Caddy) for TLS + serve SPA + route /api → :5000
- ☐ Set strong secrets: JWT_*, Postgres & RabbitMQ credentials
- ☐ Keep 5432 / 5672 / 6379 / 15672 private (VPN/allowlist for the mgmt UI)
- ☐ Run DB bring-up sequence (migrate → sync-config → load-ref → seed)
- ☐ Process manager with restart-on-failure for node index.js
- ☐ Scheduled pg_dump (or managed snapshots) shipped off-host
- ☐ Persistent volumes for Postgres and RabbitMQ

Summary. PRISM is a conventional, stateless Node + Postgres web app with a message broker and optional helpers. For the pilot it runs happily on **one right-sized VM**; the only real must-haves are **protecting Postgres (backups)** and **shipping the Chromium/WeasyPrint system libraries**. Everything else is standard three-tier hosting with a clear, low-cost path to scale when the rollout grows.